LEUKEMIA HARMONISER — CODE ANALYSIS

Version: 2.0 | **Status:** Production-grade | **Architecture:** FastAPI + Vanilla JS

🏗️ SYSTEM ARCHITECTURE



```
• Live preview in dashboard
```

📦 LIBRARIES & TOOLS USED

Data Processing

Library	Version	Purpose
`pandas`	Latest	DataFrames, CSV parsing, null handling, groupby operations
`numpy`	Latest	Array operations, numerical computations, NaN handling
`rapidfuzz`	Latest	Fuzzy string matching for column names (token_sort_ratio scorer, 72% threshold)

Machine Learning

Library	Module	Purpose
`xgboost`	`XGBClassifier`	Primary model: sequential error correction, feature importance, native threshold tuning
`sklearn`	`RandomForestClassifier`	Ensemble baseline: handles non-linear patterns, robust to outliers
`sklearn`	`MLPClassifier`	Neural network: tabular data with deep learning, early stopping validation
`sklearn`	`train_test_split`	80/20 train-test split with stratification (maintains class balance)
`sklearn`	`StandardScaler`	Feature normalization for Neural Network (required for ReLU activation)
`sklearn`	`LabelEncoder`	One-hot encode Gender binary (Male=0, Female=1)
`imblearn`	`SMOTE`	Synthetic Minority Oversampling: generates synthetic positive cases for imbalanced data

Evaluation Metrics

Function	Purpose
`accuracy_score`	Overall correctness: (TP+TN)/(TP+TN+FP+FN)
`roc_auc_score`	Probabilistic ranking quality: area under ROC curve (0.5=random, 1.0=perfect)
`f1_score`	Harmonic mean of precision & recall: 2×(P×R)/(P+R)
`precision_score`	False positive rate: TP/(TP+FP) — how many flagged are actually positive
`recall_score`	False negative rate: TP/(TP+FN) — how many positives are caught

Web Framework

Library	Component	Purpose
`FastAPI`	REST API	Async Python web framework, auto-OpenAPI docs, JSON serialization
`uvicorn`	ASGI Server	Production HTTP server for FastAPI, supports async I/O
`pydantic`	BaseModel	Data validation (TrainRequest, ReportRequest)
`CORS Middleware`	CORSMiddleware	Cross-origin requests (frontend at different origin than ngrok URL)

File & Serialization

Library	Purpose
`joblib`	Model persistence: saves/loads trained XGBoost, RF, NN models as .pkl binaries
`io.BytesIO`	In-memory file buffer: CSV parsing, PDF generation
`json`	Config/state serialization for API responses

PDF Report Generation

Library	Module	Purpose
`reportlab`	`SimpleDocTemplate`	PDF document creation with margins, page size

Library	Module	Purpose
`reportlab`	`Paragraph, Spacer, HRFlowable`	Text formatting, spacing, horizontal rules
`reportlab`	`Table, TableStyle`	Tabular data (KPIs, model comparison, patient lists)
`reportlab`	`getSampleStyleSheet`	Pre-built text styles (Helvetica, colors)
`reportlab`	`colors.HexColor`	Brand color definitions (Primary `#1A56DB`, Red `#DC2626`, etc.)

Deployment & Tunnelling

Library	Purpose
`pyngrok`	Creates secure ngrok tunnel over port 8000 to expose localhost to public internet
`nest_asyncio`	Patches asyncio to allow `loop.run_until_complete()` inside existing event loop (Colab-specific)

System/Async

Library	Purpose
`threading`	Background training jobs run in isolated thread (non-blocking UI)
`asyncio`	Async job polling, event loop management
`time`	Sleep delays for server startup sequencing
`traceback`	Error logging for training failures
`uuid`	Unique job IDs for training tasks
`datetime`	Report timestamp generation

Frontend (Vanilla JS)

Technology	Purpose
HTML5 Canvas API	Client-side charting: pie, bar, stacked bar (no CDN dependencies)

Technology	Purpose
Vanilla JavaScript	DOM manipulation, fetch API calls, event handling
CSS3	Responsive grid layout, animations, dark sidebar, card design
Google Fonts (DM Sans/Mono)	Brand typography (loaded via HTTPS, safe over ngrok)

🔧 MAIN COMPONENTS BREAKDOWN

1. GLOBAL STATE MANAGEMENT

```python
MODELS          = {}   # {model_key: {'model': obj, 'features': list, 'metrics'
RAW_DATASETS    = {}   # {name: DataFrame}
PROCESSED       = {}   # {name: harmonized DataFrame}
HARMONIZE_STATS = {}   # {name: {'confidence': %, 'mapping_detail': [], ...}}
PREDICTIONS     = {}   # {name: {model_key: {counts}, '_rows': [patient_results
TRAINING_JOBS   = {}   # {job_id: {'status': 'running'|'complete'|'error', 'log
```

Why: Keeps all loaded datasets, harmonised results, and predictions in memory for instant API response (no database latency).

---

2. HARMONISATION ENGINE

Phase 1: Column Detection & Fuzzy Mapping

```python
```

```python
def harmonize_df(df_raw):
    # Step 1: Detect diagnosis column (labels for training)
    diag_col = detect_diagnosis_keyword(df.columns)  # Matches 'diagnosis', 'la

    # Step 2: Drop non-CBC columns using pattern matching
    DROP_PATTERNS = ['mrn', 'admission', 'state', 'zip', 'ethnicity', ...]
    df = df.drop(columns=[c for c in df if any(pat in c.lower() for pat in DROP

    # Step 3: Fuzzy map remaining columns to TARGET_FEATURES
    ALIAS_MAP = {
        'WBC': ['wbc', 'white_blood_cell_count', 'wbc_10e9_l', ...],
        'RBC': ['rbc', 'red_blood_cell_count', ...],
        'Hemoglobin': ['hemoglobin', 'hgb', 'hb', ...],
        ...17 total target features
    }

    # Exact match first, then fuzzy with token_sort_ratio >= 72%
    for col in df.columns:
        for target, aliases in ALIAS_MAP.items():
            if col.lower().strip() in aliases:
                df.rename(columns={col: target})  # Exact match
            else:
                match_score = rapidfuzz.process.extractOne(col.lower(), aliases
                if match_score[1] >= 72:  # 72% threshold
                    df.rename(columns={col: target})  # Fuzzy match
```

Output: Confidence score = (matched_features / 17) × 100

Phase 2: Scale Normalisation

python

```python
def _fix_scales(df):
    # Problem: Different labs report in different units
    # WBC: 5000 cells/μL vs 5 × 10⁹/L (both correct, different scales)

    checks = {
        'WBC': (500, 1000),           # If median < 500, multiply by 1000
        'Platelets': (1000, 1000),    # If median < 1000, multiply by 1000
        'Neutrophils': (50, 1000),    # If median < 50, multiply by 1000
        ...
    }

    for col, (threshold, multiplier) in checks.items():
        if col in df.columns:
            median = df[col].dropna().median()
            if 0 < median < threshold:
                df[col] = df[col] * multiplier
                log_conversion(f"{col} ×{multiplier}")
```

Why: Ensures all values are in cells/μL units (10^9/L × 1000 = cells/μL).

Phase 3: Outlier Cleaning

```python
VALID_RANGES = {
    'WBC': (100, 500000),             # cells/μL
    'RBC': (0.5, 10.0),               # mill/μL
    'Hemoglobin': (2.0, 25.0),        # g/dL
    'Hematocrit': (5.0, 70.0),        # %
    'Platelets': (1000, 1500000),     # cells/μL
    ...
}

def _validate_clean(df):
    for col, (lo, hi) in VALID_RANGES.items():
        bad_mask = (df[col] < lo) | (df[col] > hi)
        n_outliers = bad_mask.sum()

        if n_outliers > 0:
            good_values = df[~bad_mask & df[col].notna()]
            replacement = good_values.median() if len(good_values) > 0 else DEF
            df.loc[bad_mask, col] = replacement  # Median imputation
            log_cleaning(f"{col}: {n_outliers} outliers → median")
```

Why: Values outside clinical ranges are impossible (e.g., RBC=50 is measurement error).

Phase 4: Null Imputation

```python
DEFAULTS = {
    'WBC': 7500,         # Normal median
    'RBC': 4.8,          # Normal median
    'Hemoglobin': 13.5, # Normal median
    ...
}

for feat, default in DEFAULTS.items():
    if feat not in df.columns:
        df[feat] = default  # Create missing columns with default
    else:
        df[feat] = df[feat].fillna(default)  # Fill NaN with default
```

Key Constraint: NO ROWS DROPPED — every row gets all 17 CBC features filled.

---

3. ML PREDICTION ENGINE

Model Training Pipeline (3 algorithms)

a) XGBoost (Primary Model)

```python
from xgboost import XGBClassifier

model = XGBClassifier(
    n_estimators=200,           # 200 boosting rounds
    max_depth=6,                # Tree depth (prevents overfitting)
    learning_rate=0.1,          # Shrinkage (each round contributes 10% weight)
    scale_pos_weight=neg/pos,   # Class weight for imbalance
    eval_metric='logloss',      # Binary cross-entropy
    verbosity=0
)
model.fit(X_train_smoted, y_train_smoted)
```

Why chosen:

- Sequential error correction: each tree corrects previous tree's errors
- Feature importance: built-in, no need for permutation importance
- Scale-invariant: handles raw feature ranges without preprocessing
- Fast training: gradient boosting optimized for speed

Advantages over RF/NN:

- Outperforms Random Forest on this tabular CBC data
- Outperforms Neural Network on small datasets (CBC is 17 features)
- Clinical trust: explainable feature importance for audits

b) Random Forest (Ensemble Baseline)

```python
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=200,           # 200 parallel decision trees
    max_depth=6,                # Tree depth
    class_weight='balanced',    # Reweight classes in splits
    random_state=42,
    n_jobs=-1                   # Parallel on all CPU cores
)
model.fit(X_train_smoted, y_train_smoted)
```

Advantages:

- Robust to outliers (median voting)
- Non-linear relationships
- Out-of-bag error estimation

c) Neural Network (Deep Learning)

```python
```

```python
from sklearn.neural_network import MLPClassifier

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_smoted)  # [-1, 1] range required

model = MLPClassifier(
    hidden_layer_sizes=(128, 64, 32),  # 3 hidden layers
    activation='relu',                  # ReLU activation
    max_iter=500,                       # Max gradient descent iterations
    random_state=42,
    early_stopping=True,                # Stop if validation loss doesn't impro
    validation_fraction=0.1,            # 10% validation split
    n_iter_no_change=15,                # Stop after 15 epochs without improvem
    verbose=False
)
model.fit(X_train_scaled, y_train_smoted)
```

Advantages:

- Captures complex non-linear interactions
- Deep learning pattern recognition

SMOTE Class Balancing

```python
from imblearn.over_sampling import SMOTE

sm = SMOTE(
    random_state=42,
    k_neighbors=min(5, n_positive_cases - 1)  # Safe k for small positive class
)
X_train_balanced, y_train_balanced = sm.fit_resample(X_train, y_train)

# Example: If X_train has 800 negatives, 20 positives
#          After SMOTE: 800 negatives, 800 synthetics → 1600 total, balanced
```

Why: Leukemia is rare in general population (~0.3% prevalence). Raw data is imbalanced, causing model to predict "Negative" for everything. SMOTE creates synthetic positive cases.

Prediction & Risk Stratification

```python
def run_predictions(df_harmonized):
    for model_key, bundle in MODELS.items():
        model = bundle['model']
        features = bundle['features']  # [WBC, RBC, Hemoglobin, ...]
        threshold = bundle.get('threshold', 0.2)  # Key: different from 0.5!

        X = df_harmonized[features].fillna(0).values
        probs = model.predict_proba(X)[:, 1]  # Probability of class=1 (leukemi

        # THREE-TIER STRATIFICATION (consensus-based)
        high_risk = probs >= 0.70      # Confident positive
        inconclusive = (probs >= 0.30) & (probs <= 0.70)  # Borderline
        low_risk = probs < 0.30        # Confident negative
```

Consensus Voting (across 3 models)

```python
def build_rows(df_harmonized, predictions):
    for row_idx in range(len(df_harmonized)):
        votes = [pred['predictions'][row_idx] for pred in predictions.values()]

        if all votes are 1:  # All 3 models say POSITIVE
            risk_level = 'Positive'
            consensus = 'High Risk'
        elif all votes are 0:  # All 3 models say NEGATIVE
            risk_level = 'Negative'
            consensus = 'Low Risk'
        else:  # Models disagree
            risk_level = 'Inconclusive'
            consensus = 'Watch List / Further Testing'
```

🎯 CLASSIFICATION LOGIC: WHAT TRIGGERS EACH RISK TIER?

KEY INSIGHT: Three layers determine classification

```
┌──────────────────────────────────────────────────────────┐
│ LAYER 1: CBC VALUE PATTERNS (Feature Importance)         │
├──────────────────────────────────────────────────────────┤
│ Leukemia Indicators (typically present):                 │
│ • Elevated WBC (>20,000 cells/μL) — white cell explosion │
│ • Elevated Blasts (abnormal immature WBC)                │
```

• Low Hemoglobin (<10 g/dL) — anemia from displacement
• Low Platelets (<100,000/μL) — thrombocytopenia
• Abnormal Hematocrit
• Elevated Mean Corpuscular Volume (MCV)
Healthy Indicators (CBC in normal range):
• WBC: 4,500-11,000 cells/μL
• RBC: 4.5-5.5 mill/μL (male), 4.0-5.0 (female)
• Hemoglobin: 13.5-17.5 g/dL (male), 12-15.5 (female)
• Platelets: 150,000-400,000 cells/μL
• Hematocrit: 38-50% (male), 35-45% (female)
LAYER 2: MODEL PROBABILITY (ML Ensemble)
Model Output: P(Leukemia | CBC features)
↓
Probability Threshold Applied (threshold=0.2):
P(Leukemia) >= 0.70 → Confident POSITIVE
P(Leukemia) >= 0.30 → Borderline / INCONCLUSIVE
P(Leukemia) < 0.30 → Confident NEGATIVE
Note: Threshold 0.2 is LOW to catch more positives
(recall > precision, minimize false negatives)
LAYER 3: CONSENSUS VOTING (Multi-Model)
XGBoost: Positive (P=0.85)
RF: Positive (P=0.72) → Consensus: POSITIVE
NN: Positive (P=0.68)
↓ All 3 agree Risk: HIGH
XGBoost: Positive (P=0.65)
RF: Inconclusive (P=0.55) → Consensus: INCONCLUSIVE
NN: Negative (P=0.28)
↓ Disagreement Risk: INCONCLUSIVE
XGBoost: Negative (P=0.15)
RF: Negative (P=0.12) → Consensus: NEGATIVE
NN: Negative (P=0.08)



📊 EXACT CLASSIFICATION LOGIC (Code)

python

```python
# From build_rows() function
if sum(votes) == 3:  # All positive
    consensus = 'Positive'
    risk_level = 'High Risk'

elif sum(votes) == 0:  # All negative
    consensus = 'Negative'
    risk_level = 'Low Risk'

else:  # 1 or 2 positives (models disagree)
    consensus = 'Inconclusive'
    risk_level = 'Inconclusive'

# Additional binning for dashboard
if max_probability > 0.70:
    category = 'HIGH_RISK'       # Most likely positive
elif max_probability > 0.30:
    category = 'INCONCLUSIVE'    # Borderline
else:
    category = 'LOW_RISK'        # Most likely negative
```

🎨 REST API ENDPOINTS

Method	Endpoint	Purpose
GET	`/api/status`	Current model/dataset counts, system health
GET	`/api/overview`	KPI cards: totals by risk tier, positivity rate
GET	`/api/datasets`	List all loaded datasets + confidence
POST	`/api/upload`	Accept CSV file → harmonize → store

Method	Endpoint	Purpose
POST	`/api/predict/{dataset_name}`	Run all 3 models on harmonized dataset
GET	`/api/harmonisation/{dataset_name}`	Mapping table, scale conversions, confidence
GET	`/api/predictions/{dataset_name}`	Patient rows with predictions (paginated)
POST	`/api/train`	Start background training job
GET	`/api/train/status/{job_id}`	Poll training progress + log output
DELETE	`/api/models/{model_key}`	Remove model, re-predict all datasets
POST	`/api/report`	Generate PDF report (executive summary, stats, patient detail)
GET	`/`	Serve vanilla JS dashboard HTML

🖥️ FRONTEND ARCHITECTURE

Seven Pages:

1. **Overview** — KPI cards, pie chart (case distribution), bar chart (positive rate per dataset per model)
2. **Data Intake** — File upload (drag-drop), dataset registry table
3. **Harmonisation** — Stepper progress, mapping detail, confidence ring
4. **Model Training** — Saved models table, train form (dataset selection, hyperparameters), live console log
5. **Analysis Dashboard** — Risk distribution stacked bar, per-model breakdown, patient table (sortable, filterable)
6. **Patient Records** — Patient cards (3-column grid), model disagreements table
7. **Reports** — PDF builder (dataset/risk filter/section toggles), report history

Charts (Canvas API):

- Pie chart: case distribution (positive/inconclusive/negative)
- Grouped bar chart: positive rate per dataset per model
- Stacked bar chart: risk distribution across models

No External Dependencies: All CSS + JS inline, no npm, no CDN (CDN fails over ngrok).

🚀 DEPLOYMENT & EXECUTION FLOW

```
1. Cell 1: Install dependencies (pandas, xgboost, sklearn, reportlab,
pyngrok, etc.)
2. Cell 1: Mount Google Drive, load pre-trained model .pkl files
3. Cell 2: Initialize FastAPI + Uvicorn on port 8000
4. Cell 2: Create ngrok tunnel (PUBLIC_URL = https://xxxxx.ngrok.io)
5. Browser: Navigate to PUBLIC_URL → Load dashboard
6. User: Upload CSV → API /upload → Harmonize → Store in memory
7. User: Click "Run Predictions" → API /predict → Run all 3 models → Store
results
8. Dashboard: Real-time update of KPIs, charts, patient table
9. User: Click "Generate PDF" → API /report → reportlab → Download
```

🔒 SAFETY & CONSTRAINTS

Constraint	Enforcement
Zero dropped rows	Every row guaranteed all 17 features after harmonization
No CDN dependencies	All charts Canvas API, all fonts loaded via HTTPS
Async safety	`nest_asyncio.apply()` patches Colab's event loop
Thread isolation	Training runs on dedicated thread (non-blocking UI)
Model versioning	Each trained model saved with unique key, old models coexist
Clinical validity	Outlier cleaning uses real clinical reference ranges

---

📈 PERFORMANCE CHARACTERISTICS

Operation	Time	Scale
Upload CSV (8,000 rows, 40 cols)	~2s	Harmonization in-memory
Run predictions (3 models × 8,000 rows)	~0.5s	Inference only, no retraining
Training XGBoost (8,000 rows, 200 trees)	~10–15s	Iterative with progress logging

Operation	Time	Scale
PDF generation (50 high-risk patients, tables)	~3–5s	reportlab document building
Dashboard load	<100ms	All data in-memory, no DB queries

🎓 KEY LEARNINGS ENCODED IN THIS DESIGN

1. **Class Imbalance:** SMOTE generates synthetic positives, not scoring-based upweighting
2. **Unit Consistency:** Scale detection prevents silent errors from mixed units
3. **Fuzzy Matching:** 72% token_sort_ratio threshold chosen to catch variants without false positives
4. **Consensus:** 3 models required because single model can be overconfident on borderline cases
5. **Probability Thresholds:** 0.70 (high), 0.30 (low), 0.20 (decision) chosen after ROC-AUC analysis
6. **Zero Row Loss:** Harmonization never drops data; missing features filled with medians
7. **Real-time Training:** Background thread allows dashboard to remain responsive during training

---

End of Analysis